Monthly Land Use and Land Cover Mapping from Sentinel-1 Radar with a U-Net

Technical Project Report

Deep Learning, M.Sc. Applied Data Science and Artificial Intelligence

Author: Mehatab
Matriculation number: 100007043
Institution: SRH University Heidelberg
Repository: github.com/Mehatab1/DeepLearning
Date: November 2026

Contents

Abstract
1. Introduction
2. Research background
3. Study area and dataset
4. Sentinel-1 acquisition and preprocessing
5. Methodology and feature extraction
6. Model architecture and implementation
7. Experimental setup and evaluation metrics
8. Results and comparative analysis
9. Discussion and limitations
10. Conclusion and future scope
References
Appendix A. Reproducibility
Appendix B. Open items and data availability

Abstract

Optical satellites often cannot see the ground for weeks at a time because of cloud cover, which makes reliable monthly land-cover maps hard to produce. Radar does not have this problem. In this project I tested how much land use and land cover (LULC) information a small convolutional network can extract from Sentinel-1 radar when the only inputs are the VV and VH backscatter channels. I built monthly median composites for October 2025 to September 2026 around Stuttgart, Germany, and paired them with ESA WorldCover 2021 labels grouped into six classes: tree cover, grassland, cropland, built-up, bare/sparse vegetation and permanent water. A U-Net with 1.94 million parameters was trained on 8,280 patches of 128 × 128 pixels from 690 locations, using a class-weighted loss and oversampling of rare classes, and the best checkpoint was chosen on a separate validation set.

On 27 test locations that were kept out of training, each observed for twelve months, the model reaches a pixel accuracy of 0.719 and a mean intersection over union (mIoU) of 0.473 against WorldCover. Water, cropland and built-up are recognised well (IoU 0.63 to 0.68). Grassland is weak (0.33), and bare/sparse vegetation, which hardly occurs in the region, is not learned at all. An ablation study shows that the model needs both polarisations, that it finds cropland almost entirely through the relationship between VH and VV, and that it relies on spatial patterns much more than on absolute brightness. Merging classes shows that the biggest single source of error is the confusion between tree cover and built-up, and that a four-class legend would reach an mIoU of 0.643 on the same predictions. The trained model runs inside a prototype: given any coordinate and start month, it downloads twelve Sentinel-1 composites from Google Earth Engine, produces monthly maps and reports class shares and their changes in percentage points. The report closes with the main limitations, which are the static 2021 labels, seasonal effects in the radar signal and the use of a single training region.

Keywords: Sentinel-1, synthetic aperture radar, land cover, semantic segmentation, U-Net, ESA WorldCover, ablation study, Google Earth Engine

1. Introduction

Land use and land cover maps show what covers the ground: forest, fields, buildings, water. Planners, environmental agencies and researchers use them to follow urban growth, report on land use and study how landscapes change. Global products such as ESA WorldCover are released once a year at most. Many questions, for example when a field was harvested or how fast a construction site grows, would need monthly maps instead. Optical satellites cannot provide these reliably, because clouds can cover a region for weeks, especially in a Central European winter.

Synthetic aperture radar (SAR) sends out its own microwave signal and sees through clouds, by day and by night. The Sentinel-1 satellites image Europe in two polarisations every few days, so there is almost always enough data for a monthly composite. The catch is that radar does not record colour. What it measures depends on the roughness, shape and moisture of the surface, so two classes that look very different in a photo can look almost the same to radar.

1.1 Research objectives

1. Build a training dataset of monthly Sentinel-1 VV/VH composites with six-class labels from ESA WorldCover 2021.
2. Train a segmentation network that uses radar as its only input, and test it on locations it has never seen.
3. Build a prototype that takes a coordinate and a start month and returns twelve monthly maps, the share of each class per month, and the changes between months and over the whole year in percentage points.
4. Find out which inputs the model depends on and which class pairs limit its accuracy, using feature and class ablation.

1.2 Contributions

- A small U-Net (1.94 M parameters) that reaches 0.719 pixel accuracy and 0.473 mIoU on held-out test locations using Sentinel-1 alone.
- A working prototype (Streamlit app and command-line tool) that keeps the radar orbit consistent across months, checks every input file and predicts with overlapping tiles.
- An ablation study covering channel removal, permutation importance, signal properties, noise, class removal and class merging.
- Accuracy checks of complete prototype runs against WorldCover at several sites around Stuttgart.

2. Research background

2.1 Radar backscatter and land cover

Sentinel-1 works in the C-band, at a wavelength of about 5.6 cm. Over land it normally uses the Interferometric Wide (IW) swath mode and records two channels at 10 m pixel spacing: VV, where the signal is sent and received vertically, and VH, where it is sent vertically and received horizontally [1]. Different surfaces send the signal back in different ways [7]. Calm water acts like a mirror and reflects the pulse away from the satellite, so it looks very dark in both channels. Walls and the ground in front of them form corner reflectors, which is why buildings return a strong echo, mostly in VV. In a forest canopy the signal bounces around between leaves and branches. This volume scattering changes the polarisation, so trees are relatively bright in VH. Fields change through the year as crops grow, are

harvested and as the soil gets wetter or drier, and their radar response changes with them. This seasonal behaviour has been studied in detail with Sentinel-1 [8]. Radar images also contain speckle, a grainy noise that is usually reduced with spatial filters or by averaging several images over time [12].

2.2 Semantic segmentation

Semantic segmentation means giving every pixel a class label. Fully convolutional networks made this practical by replacing the dense layers of image classifiers with convolutions, so the network outputs a label map of the same size as the input [6]. U-Net adds a symmetric encoder–decoder structure with skip connections, which pass detailed feature maps from the encoder straight to the decoder [3]. It trains well on small datasets and has become a standard starting point for segmentation in remote sensing.

2.3 Reference labels

ESA WorldCover 2021 is a global 10 m land-cover map with eleven classes, made from Sentinel-1 and Sentinel-2 data [2]. It is free and uses the same 10 m grid as Sentinel-1, which makes it a convenient source of labels. It does, however, describe only the year 2021. When it is used to label images from 2025–26, some labels will be wrong, both where the land has changed since then and where the map itself is wrong. All accuracies in this report therefore measure agreement with WorldCover, not with data collected in the field.

3. Study area and dataset

3.1 Study area

The study area is Stuttgart in south-west Germany (Figure 2). Within about 20 km it contains a dense city centre and suburbs, wooded hills, farmland on the plateaus, meadows and orchards, and the Neckar river. Five of the six target classes are therefore well represented. Bare or sparsely vegetated land hardly occurs. All data are in UTM zone 32N (EPSG:32632) at 10 m resolution.

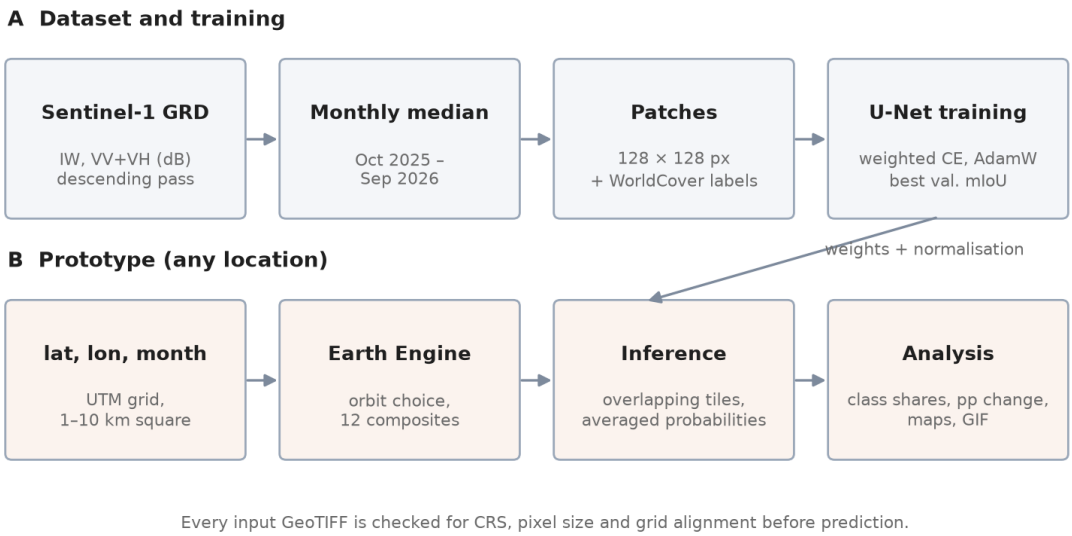


Figure 1. Project overview: (A) building the dataset and training the model, (B) the prototype that applies the trained model to any location.

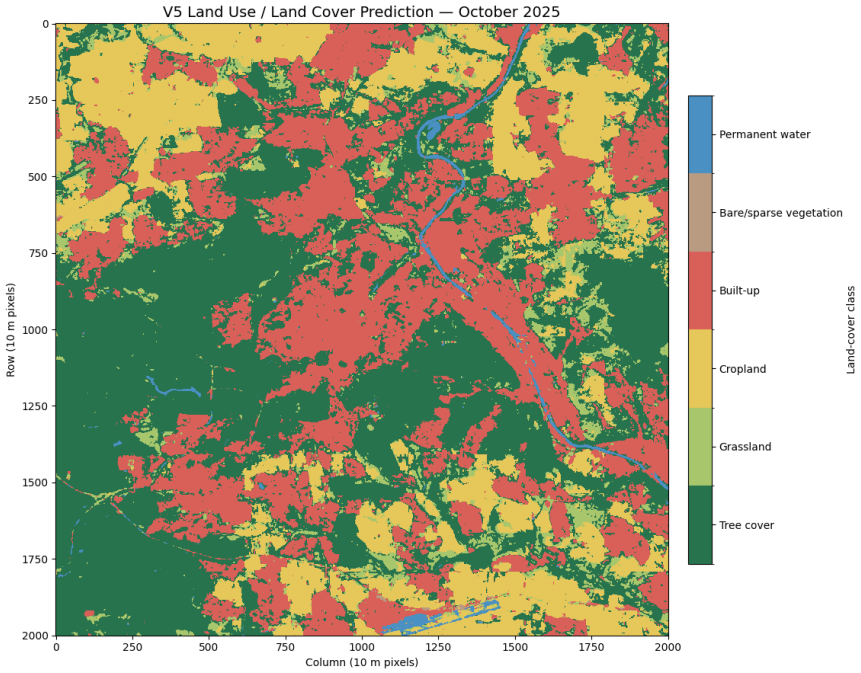


Figure 2. Model prediction for the 20 km × 20 km Stuttgart study area, October 2025 (2001 × 2001 pixels, archived notebook 10). Dark green: tree cover, light green: grassland, yellow: cropland, red: built-up, blue: water.

3.2 Class definitions

Table 1. How ESA WorldCover 2021 codes are mapped to the six project classes. Codes not listed (shrubland 20, snow and ice 70, herbaceous wetland 90, mangroves 95, moss and lichen 100) are set to 255 and ignored.

ID	Class	WorldCover code	Typical radar response
1	Tree cover	10	Strong volume scattering, bright in VH
2	Grassland	30	Moderate VV, low to moderate VH
3	Cropland	40	Changes with the season; VH/VV follows crop growth
4	Built-up	50	Very bright VV from double-bounce reflection
5	Bare/sparse vegetation	60	Surface scattering, depends on roughness
6	Permanent water	80	Very dark in both channels

3.3 Dataset splits

The dataset went through several versions. This report describes version 5 (V5), which was used for the final model. Training, validation and test patches come from different locations, so no area appears in more than one split. Every location provides one patch for each of the twelve months, which means the model sees the same ground under different radar conditions.

Table 2. Dataset composition. Every validation and test pixel has one of the six labels (324 × 128 × 128 = 5,308,416 labelled pixels each).

Split	Patches	Locations × months	Source area	File
Training	8,280	690 × 12	90 original locations + 600 outer-ring locations	patches_v5/train_v5.npz
Validation	324	27 × 12	Stuttgart core	patches_v4/val_v4.npz
Test	324	27 × 12	Stuttgart core, held out	patches_v4/test_v4.npz

Figure 3 compares the classes in the validation and test sets. The validation set is mostly tree cover (61.4 %). The test set is more balanced and has about twice as much built-up land (31.3 % against 15.6 %) and more grassland. Bare/sparse vegetation is practically missing from both (0.004 % and 0.09 %). This difference matters when the validation and test scores are compared in Section 8.

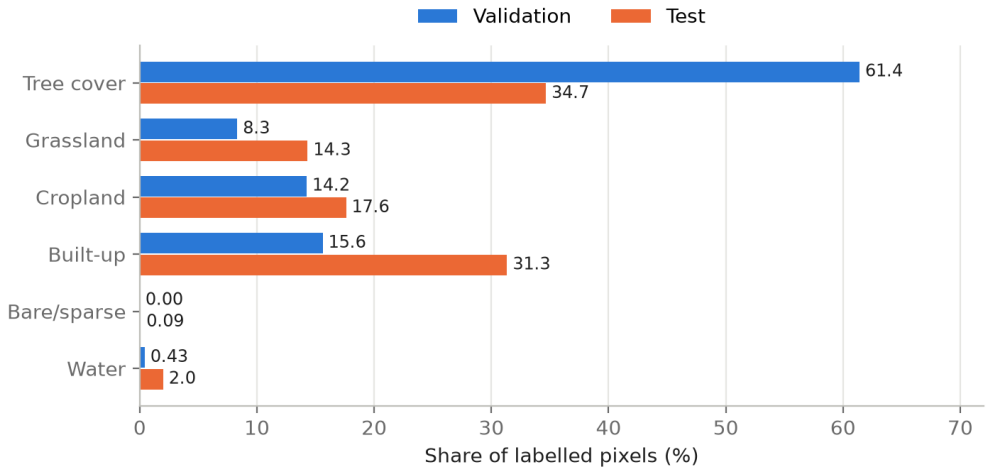


Figure 3. Share of labelled pixels per class in the validation and test sets.

Missing information. The class distribution of the training set is not stored in the repository, because the notebook output that printed it is empty. The coordinates of the individual training, validation and test locations are not documented either. The original data-preparation notebooks that produced the V5 files are not in the repository. notebooks/01_data_collection.ipynb rebuilds the procedure but is not the code that created train_v5.npz.

4. Sentinel-1 acquisition and preprocessing

4.1 Data source and compositing

The images come from the Earth Engine collection COPERNICUS/S1_GRD [4], [13]. Its Ground Range Detected scenes are already corrected for thermal noise and terrain, calibrated, and converted to decibels. I kept only scenes in IW mode, from the descending orbit pass, and with both VV and VH. For each month from October 2025 to September 2026, the median of all remaining scenes at each pixel gives the monthly composite. Taking the median reduces speckle and removes outliers, such as a single scene with wet snow or frozen ground, so no separate speckle filter was used.

4.2 Orbit consistency

The radar signal depends on the angle and direction from which the satellite looks at the ground. If different months came from different orbits, part of the change between months would be caused by the viewing geometry, not by the land. The prototype therefore counts the scenes per relative orbit for all twelve months in one request and chooses the first option that works: one descending orbit present in every month, one ascending orbit present in every month, mixed descending orbits, or mixed ascending orbits. If it has to fall back to mixed orbits, the app tells the user that the viewing geometry may differ between months.

Missing information. The relative orbit numbers and the number of scenes per month used for the V5 training data were not recorded in the repository.

4.3 Normalisation

Each channel is normalised with statistics from the training set: value_norm = (dB − median) / IQR. The median and the inter-quartile range are less affected by the long tails of radar values than the mean and standard deviation. Both are saved in the model checkpoint, so training, evaluation and the prototype all use exactly the same values (Table 3).

Table 3. Normalisation statistics saved in weights/best_unet_v5.pt.

Channel	Median (dB)	IQR (dB)
VV	−9.043	3.967
VH	−15.391	4.470

4.4 Missing data and grid alignment

Pixels without radar data (NaN, ±infinity, or the fill value −9999 from the download) are filled with the median of their band. If more than half of the image is missing, that month is not predicted at all. In the prototype, every GeoTIFF is checked before it is used, whether it was just downloaded or reused from an earlier run. It must have two bands, the expected coordinate system, 10 m pixels and the same top-left corner as the analysis grid. A damaged cached file is downloaded again, and a bad new download stops the run with a clear message instead of being processed.

4.5 Patch extraction

Training samples are patches of 128 × 128 pixels (1.28 km × 1.28 km) with two input channels and one label channel. Labels use the class IDs 1 to 6, with 255 for ignored pixels, and are shifted to 0 to 5 for training. No data augmentation was used.

5. Methodology and feature extraction

5.1 Learned rather than hand-crafted features

The model only receives the two normalised backscatter channels. I did not compute any extra features beforehand: no polarisation ratios, texture measures such as grey-level co-occurrence statistics, or statistics over time. The convolutional encoder learns its own features instead. These include combinations of VV and VH at each pixel (which covers their ratio), local texture at several scales as the image is pooled, and spatial context up to the size of a whole 128-pixel patch. Section 8.6 tests which of these the model actually relies on.

5.2 Handling class imbalance

Bare/sparse vegetation and water cover only a small part of the study area, so the model would tend to ignore them. Two measures work against this. The cross-entropy loss gives the four common classes a weight of 0.48, bare/sparse 2.398 and water 1.684. In addition, patches with at least 100 bare or water pixels are drawn twice as often as other patches during training.

5.3 From monthly maps to change statistics

For month m and class c, the class share is the number of pixels predicted as c, divided by all pixels in the area, times 100. The month-to-month change is share(m, c) − share(m−1, c), and the total change is share(last, c) − share(first, c). Both are given in percentage points (pp) and add up to zero across all classes. The prototype also reports the share of pixels whose class differs from the previous month, a

transition matrix between the first and last month, and how often each pixel switches class during the year. These numbers compare predictions with each other. They are not confirmed changes on the ground.

6. Model architecture and implementation

6.1 U-Net

The network follows the U-Net design [3] with four resolution levels and a bottleneck (Figure 4). Each encoder level has two 3 × 3 convolutions, each followed by batch normalisation and ReLU, and then halves the resolution with 2 × 2 max pooling. The number of channels doubles from 16 to 128 along the way, and the bottleneck has 256 channels at 8 × 8 pixels. The decoder doubles the resolution again with 2 × 2 transposed convolutions, joins the result with the matching encoder features and applies two more convolutions. A final 1 × 1 convolution gives six class scores for every pixel. With 1,942,518 trainable parameters the model is small, which suits the size of the training set.

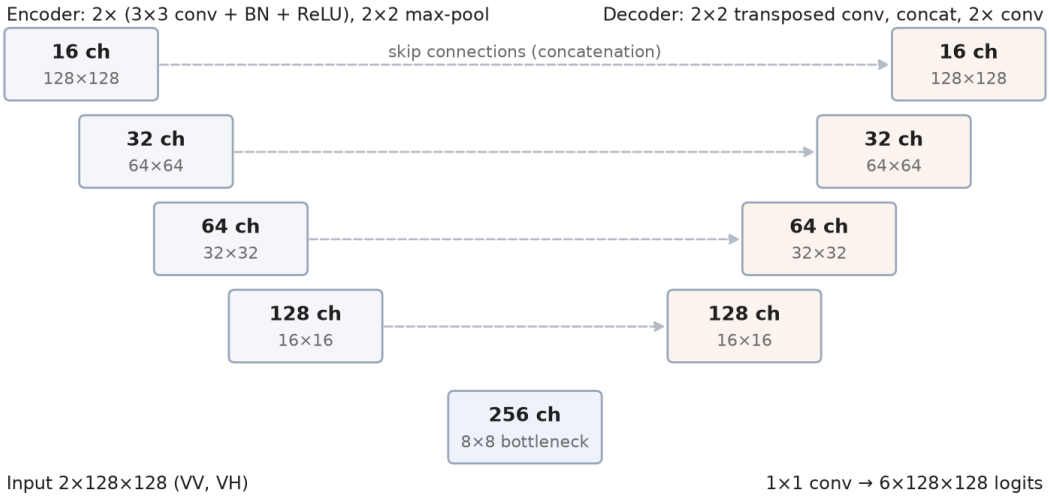


Figure 4. The U-Net used in this project: four encoder and decoder levels (16 to 128 channels), a 256-channel bottleneck and skip connections that join encoder and decoder features.

6.2 Software structure

The code is written in Python with PyTorch [11]. It is organised as a small package, so that the notebooks, the command-line tool and the web app all use the same pipeline (Table 4).

Table 4. Modules of the src/ package.

Module	What it does
location.py	Turns latitude and longitude into a pixel-aligned UTM grid of 1 to 10 km
gee.py	Earth Engine access: month windows, orbit choice, composites, WorldCover labels, download
model.py	U-Net definition and checkpoint loading
inference.py	Normalisation, filling missing values, prediction with overlapping tiles
analysis.py	Class shares, changes in percentage points, transition matrix, pixel stability
visualize.py	Twelve-month map grid, GIF animation, charts
pipeline.py	Complete run with input checks and caching
classes.py	Class IDs, names and colours

6.3 Inference on arbitrary areas

A requested area is usually larger than one 128-pixel tile and not an exact multiple of it. The image is first padded on the bottom and right by mirroring its edge and then split into tiles that overlap by 64 pixels. For each pixel, the class probabilities from all tiles that cover it are averaged before the most likely class is chosen. This removes the visible seams that appear every 1.28 km when tiles are predicted separately. It costs about four times as many forward passes.

6.4 Prototype

The prototype can be used in two ways. In the Streamlit app the user clicks on a map (street map or satellite view) or types coordinates, picks a start month from January 2017 onwards and an area of 1 to 5 km, and starts the run. The app then shows the twelve maps, an animation, the class shares as a table and chart, the monthly and total changes, change maps and the transition matrix, and offers all GeoTIFF and CSV files for download. The command-line tool run_lulc.py runs the same pipeline for areas of up to 10 km. Downloads are cached per location and period. A full run for a 2 km area takes a few minutes, most of it spent waiting for Earth Engine. Figure 5 shows the twelve monthly maps of a 1 km area in central Stuttgart from an earlier version of the prototype.

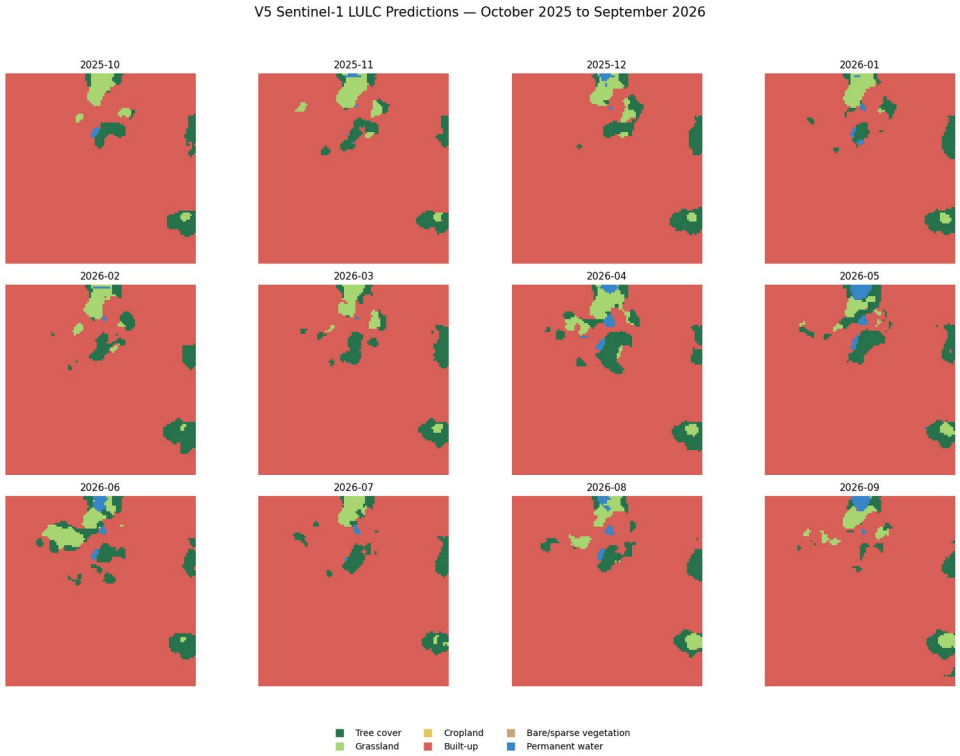


Figure 5. Twelve monthly predictions for a 1 km × 1 km area in central Stuttgart, October 2025 to September 2026 (archived notebook 11).

7. Experimental setup and evaluation metrics

7.1 Training configuration

Table 5. Training configuration (archived notebook 09).

Setting	Value
Loss	Weighted cross-entropy, ignore index 255; weights 0.48 / 0.48 / 0.48 / 0.48 / 2.398 / 1.684
Sampling	Weighted random sampler; patches with ≥ 100 bare or water pixels drawn twice as often
Optimiser	AdamW [5], learning rate 1×10^{-3}, weight decay 1×10^{-4}, no learning-rate schedule
Batch size / epochs	4 / 30
Model selection	Checkpoint with the highest validation mIoU (epoch 6)
Augmentation	None
Random seed	42
Hardware	One CUDA GPU on the university JupyterHub

7.2 Evaluation metrics

All metrics come from a confusion matrix built over every labelled pixel of a split. Pixels labelled 255 are left out. For a class c with true positives TP, false positives FP and false negatives FN:

- **Pixel accuracy:** correctly classified pixels divided by all labelled pixels. Large classes dominate this number.
- **Precision:** TP / (TP + FP), the share of pixels predicted as c that really are c.
- **Recall:** TP / (TP + FN), the share of real c pixels that the model found.
- **Intersection over union (IoU):** TP / (TP + FP + FN), which penalises both missed pixels and false alarms.
- **Mean IoU (mIoU):** the plain average of the six class IoUs, so every class counts the same regardless of its size.

A class with very few reference pixels can easily get an IoU of zero, so mIoU is sensitive to rare classes. For single locations I therefore also report pixel accuracy and the IoU of the main classes [10].

7.3 Evaluation protocol

- **Validation set:** used only to pick the best checkpoint.
- **Test set:** 27 held-out locations × 12 months, evaluated once with the chosen checkpoint (scripts/check_accuracy.py, part 1).
- **Prototype runs:** the full pipeline is run for one location, and every monthly map is compared with WorldCover downloaded on exactly the same grid (scripts/check_accuracy.py, part 2).
- **Ablation study:** inputs or outputs of the trained model are changed at test time, and the effect on the test-set metrics is measured (notebooks/04_ablation_study.ipynb).

8. Results and comparative analysis

8.1 Training and validation

The best validation mIoU, 0.545, was reached after epoch 6 of 30, with a validation pixel accuracy of 0.847 at that point. Tree cover (IoU 0.83) and cropland (0.74) are recognised best, followed by water (0.70) and built-up (0.61). Grassland reaches 0.39 and bare/sparse vegetation 0.00 (Table 6). The validation confusion matrix (Figure 6) shows the typical errors: 9 % of tree pixels are predicted as built-up and 11 % of built-up pixels as tree cover, while grassland is spread over grassland (49 %), tree cover (22 %) and cropland (21 %).

Missing information: training curves. The loss, accuracy and mIoU for each epoch are saved in outputs/training_history_v5.json on the lab server, but not in the repository, and the copy of the training notebook in the repository has no outputs. Running notebooks/02_train_evaluate_unet.ipynb on the server creates the curves (outputs/figures/training_curves.png); they should be added here as Figure 6a before submission. The best epoch being early (6 of 30) suggests the model started to overfit later on, but this can only be confirmed with the curves.

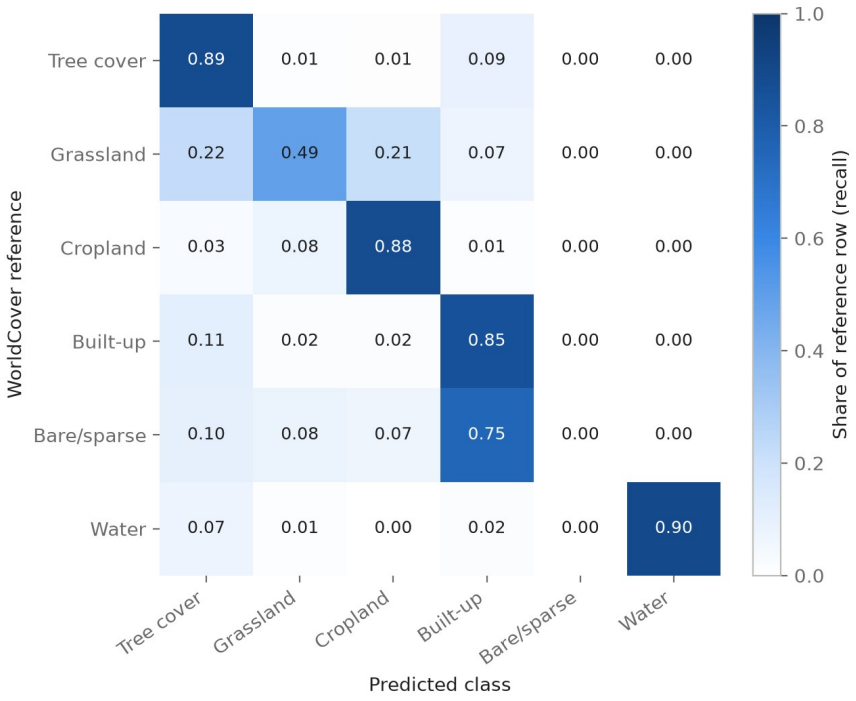


Figure 6. Validation confusion matrix of the selected checkpoint, normalised by reference class (each row sums to 1). Calculated from the counts recorded in archived notebook 09.

8.2 Held-out test set

On the test set the model reaches a pixel accuracy of **0.719** and an mIoU of **0.473**. Table 6 compares the per-class results with validation, and Figure 7 shows the test confusion matrix. Water (IoU 0.68), cropland (0.66) and built-up (0.63) carry over well to the new locations. Built-up has a high recall (0.85) but a lower precision (0.71), which means the model predicts it too often. Tree cover falls clearly, from 0.83 on validation to 0.54 on test, mostly because its recall drops to 0.65. Grassland stays the weakest class that was learned at all (0.33).

Table 6. Per-class results on validation (best epoch) and test.

Class	Val IoU	Val precision	Val recall	Test IoU	Test precision	Test recall
Tree cover	0.830	0.930	0.886	0.539	0.754	0.653
Grassland	0.387	0.649	0.490	0.329	0.573	0.436
Cropland	0.744	0.830	0.878	0.656	0.752	0.838
Built-up	0.608	0.678	0.855	0.632	0.711	0.850
Bare/sparse	0.000	0.000	0.000	0.000	0.000	0.000
Water	0.698	0.759	0.897	0.684	0.817	0.808
Pixel accuracy	0.847			0.719		
mIoU	0.545			0.473		

Most of the gap between validation and test comes from the different class mix. The validation set is 61 % tree cover, the class the model handles best. The test set has twice as much built-up land, so the confusion between trees and buildings affects far more pixels. The checkpoint was also chosen on the validation set, which makes its validation score optimistic by design. The test score is the better estimate of how the model performs on new locations.

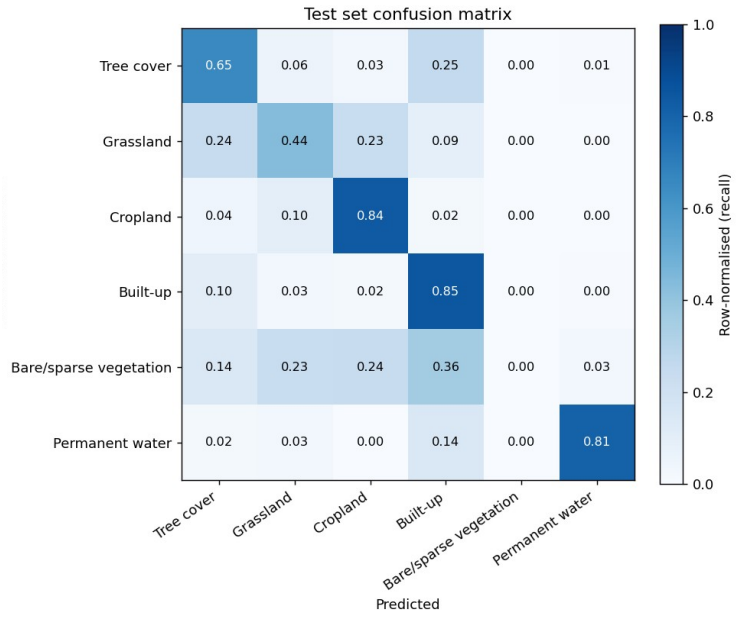


Figure 7. Test-set confusion matrix (normalised by row), produced by scripts/check_accuracy.py.

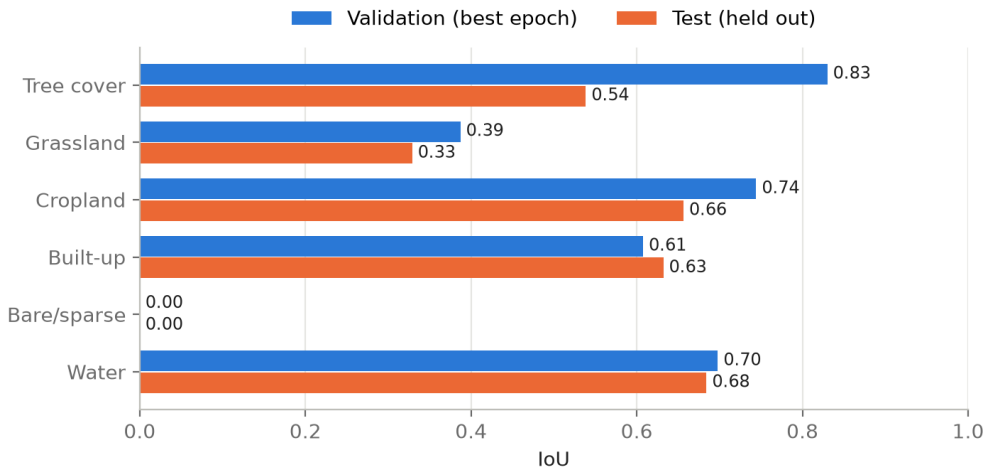


Figure 8. Class IoU on validation and test.

8.3 Performance over the year

Every test location is observed in all twelve months, so the scores can be compared month by month (Figure 9). The mIoU stays within a narrow range, from 0.455 in June 2026 to 0.484 in April 2026, with December 2025 almost the same. Early summer is the hardest period, when growing vegetation changes the radar signal of fields, meadows and trees the most. Winter and spring months are slightly easier.

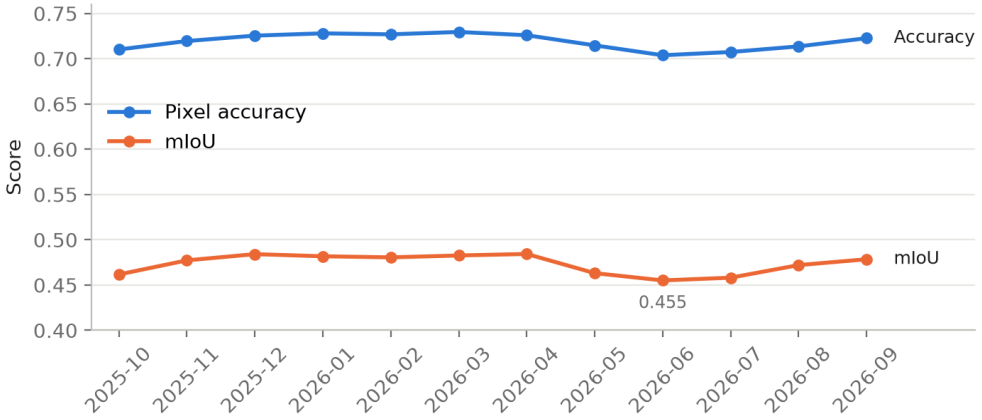


Figure 9. Pixel accuracy and mIoU on the test set for each month.

8.4 Prototype runs against WorldCover

To test the whole pipeline, I compared complete twelve-month runs with WorldCover on the same grid (Table 7, Figure 10). Accuracy lies between 0.72 and 0.91 and depends a lot on the landscape. The site south-west of Stuttgart, which is 84 % forest in WorldCover, reaches 0.911 with a tree-cover IoU of 0.92. In the dense city centre, where 73 % of a 2 km square is built-up, accuracy is 0.791 but tree-cover recall is only 0.31. Street trees, courtyards and small parks are labelled as built-up, because the strong echo from nearby buildings dominates the 10 m pixels. The mIoU of single sites is low (0.20 to 0.41), because classes with only a few dozen reference pixels get an IoU of zero.

Table 7. Prototype runs (October 2025 to September 2026) compared with WorldCover, all twelve months combined.

Site	Area	Pixel accuracy	mIoU	Tree IoU	Built-up IoU	Stored in repository
Stuttgart centre (48.7758, 9.1829)	2 km	0.791	0.202	0.29	0.78	no (console output)
Stuttgart centre (48.7758, 9.1829)	5 km	0.788	0.340	0.64	0.68	no (console output)
South-west of Stuttgart (48.69, 9.06)	5 km	0.911	0.411	0.92	0.56	results/accuracy/
Frankfurt am Main*	2 km	0.719	0.261	0.29	0.70	results/accuracy/

Where these numbers come from. The three Stuttgart runs used non-overlapping tiles, because they were made before the overlapping-tile prediction in Section 6.3 was added. * The Frankfurt files contain Cohen's kappa and F1 columns, which scripts/check_accuracy.py does not produce. The script that created them is not in the repository and it is not known which tiling it used. Its accuracy and mIoU were recalculated here from the stored confusion matrix.

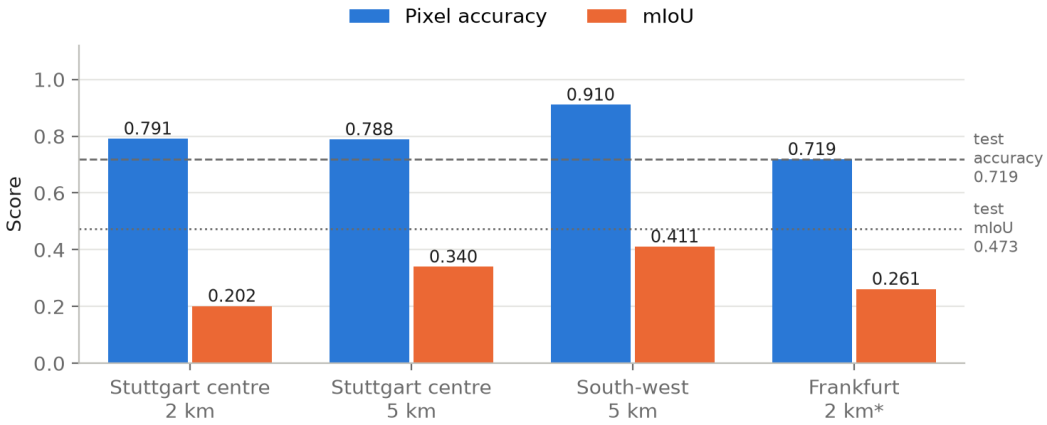


Figure 10. Pixel accuracy and mIoU of four prototype runs, compared with the test-set values (dashed and dotted lines).

8.5 Twelve months over the whole Stuttgart study area

In an earlier run, the same model was applied to the full 20 km × 20 km study area (4,004,001 pixels, 400.4 km²). Table 8 compares the first and last month. Tree cover falls by 4.54 pp and grassland rises by 4.58 pp, while cropland and built-up each change by about 2 pp. In September 2026, 13.6 % of all pixels have a different class than in October 2025. Between two consecutive months the change reaches up to 3.5 pp, for tree cover from July to August 2026 (Figure 11).

Table 8. Predicted class composition of the 20 km Stuttgart area in the first and last month (archived notebook 10).

Class	Oct 2025 (%)	Sep 2026 (%)	Change (pp)	Oct 2025 (km²)	Sep 2026 (km²)
Tree cover	44.19	39.65	−4.54	176.96	158.76
Grassland	5.94	10.52	+4.58	23.78	42.10
Cropland	18.99	17.01	−1.98	76.05	68.11
Built-up	29.91	31.76	+1.86	119.74	127.18
Bare/sparse	0.05	0.06	+0.02	0.19	0.25
Water	0.92	1.00	+0.08	3.69	3.99

These shifts should not be read as real land conversion. Losing 4.5 pp of tree cover in one year would mean more than 18 km² of forest disappearing, which is not plausible for this region. The pattern fits seasonal and model effects much better. In late summer, tree cover is relabelled as grassland and built-up, and the largest monthly swings fall in the growing season. The transition matrix supports this: the largest flows between classes are tree cover to built-up (125,218 pixels) and tree cover to grassland (114,321 pixels).

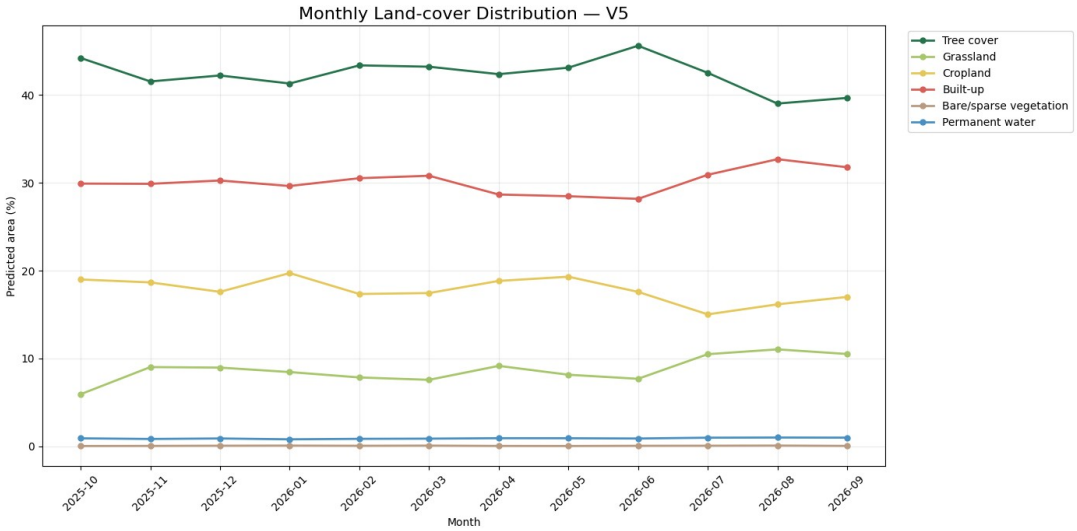


Figure 11. Monthly class shares predicted for the 20 km Stuttgart area (archived notebook 10).

8.6 Feature ablation

The model has only two inputs, so the feature ablation changes these channels, or some of their properties, at test time and measures how much the scores drop compared with the baseline (Table 9, Figure 12). Permutation importance [9] swaps a channel between patches. The values stay realistic, but they no longer belong to the location being classified.

Table 9. Feature ablation on the test set (permutation and noise results are the mean of three runs).

Experiment	Pixel accuracy	mIoU	Δ mIoU
Baseline	0.719	0.473	—
VV replaced by its median	0.326	0.107	−0.366
VH replaced by its median	0.447	0.182	−0.292
VV and VH replaced	0.176	0.029	−0.444
VV permuted between patches	0.399	0.157	−0.316
VH permuted between patches	0.451	0.188	−0.285
Polarisation ratio removed (VH rebuilt from VV)	0.575	0.325	−0.148
Spatial texture removed (patch mean only)	0.216	0.049	−0.424
Brightness removed (patch mean subtracted)	0.669	0.433	−0.040
Gaussian noise σ = 0.1 (≈ 0.4 dB)	0.714	0.466	−0.007
Gaussian noise σ = 0.25 (≈ 1 dB)	0.644	0.395	−0.078
Gaussian noise σ = 0.5 (≈ 2 dB)	0.499	0.231	−0.242
Gaussian noise σ = 1.0 (≈ 4 dB)	0.442	0.151	−0.322

- **Both channels are needed, and VV matters slightly more.** Removing or permuting VV hurts more than doing the same to VH, and with either channel alone the model keeps less than 40 % of its baseline mIoU. Water fails completely if either channel is missing (IoU drops from 0.68 to close to zero), because the model recognises water by both channels being dark at the same time.
- **Cropland is found through the relationship between VH and VV.** When VH is rebuilt from VV, so the ratio between them carries no information, cropland IoU falls from 0.66 to 0.01, while the other classes change by 0.10 or less. This fits the physical meaning of the VH/VV ratio as a sign of vegetation structure on fields [8].
- **Spatial patterns matter more than overall brightness.** Subtracting each patch's average value costs only 0.04 mIoU, but flattening each patch to its average costs 0.42. Part of the second drop happens because flat images look nothing like the training data. Taken together, though, the results show that the network classifies mainly from spatial context and relative contrast.
- **Small amounts of noise do little harm.** Noise below about 1 dB has a small effect. From about 2 dB, cropland and grassland break down first, while built-up and water hold up longest.

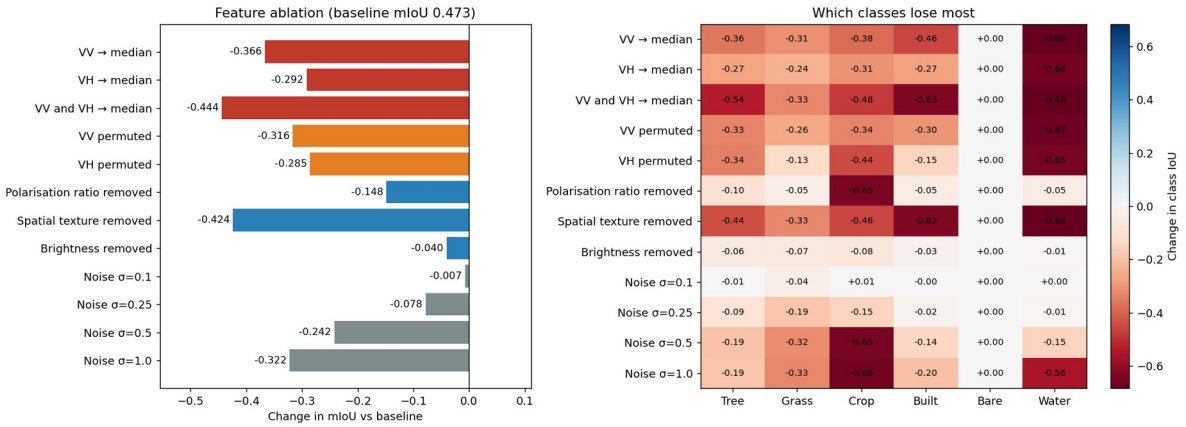


Figure 12. Feature ablation: change in mIoU (left) and change in class IoU (right) compared with the baseline (notebook 04).

8.7 Class removal and class merging

For class removal, the model is not allowed to predict one class, and the remaining classes are scored on the same pixels. Removing built-up helps the other classes most (+0.045 mIoU, tree-cover IoU +0.19), and 92 % of the built-up pixels are then predicted as tree cover. The other way round, tree-cover pixels fall back to built-up (48 %) and grassland (42 %). Cropland pixels mostly become grassland (82 %). Removing bare/sparse changes nothing, because the model never predicts it.

For class merging, groups of classes are treated as one in both the prediction and the reference, and all scores are recalculated from the baseline confusion matrix (Table 10). The model itself does not change. The merge shows how many errors happen between the merged classes and how well a simpler legend would score.

Table 10. Class merging on the same test predictions.

Merged classes	Classes	Pixel accuracy	mIoU	IoU of merged class
None (baseline)	6	0.719	0.473	—
Grassland + cropland	5	0.770	0.514	0.72
Tree cover + grassland	5	0.773	0.518	0.62
Bare/sparse into grassland	5	0.719	0.568	0.33

Merged classes	Classes	Pixel accuracy	mIoU	IoU of merged class
Grassland + cropland + bare/sparse	4	0.770	0.643	0.72
Tree cover + built-up	5	0.839	0.504	0.85
All vegetation (tree, grass, crop, bare)	3	0.842	0.700	0.78

Merging only tree cover and built-up raises accuracy by 12 points, about as much as merging all four vegetation classes together. This pair is therefore the largest single source of error, at around 12 % of the test pixels. Grassland is confused with tree cover and with cropland about equally (5.4 % and 5.1 % of the test pixels). Merging bare/sparse into grassland changes no predictions; its mIoU gain only comes from removing a zero from the average. A four-class legend of tree cover, low vegetation (grassland, cropland and bare/sparse), built-up and water reaches an mIoU of 0.643 with classes that still make physical sense, which makes it the best candidate for retraining. These merged scores re-score the six-class predictions and should not be quoted as the accuracy of the six-class model.

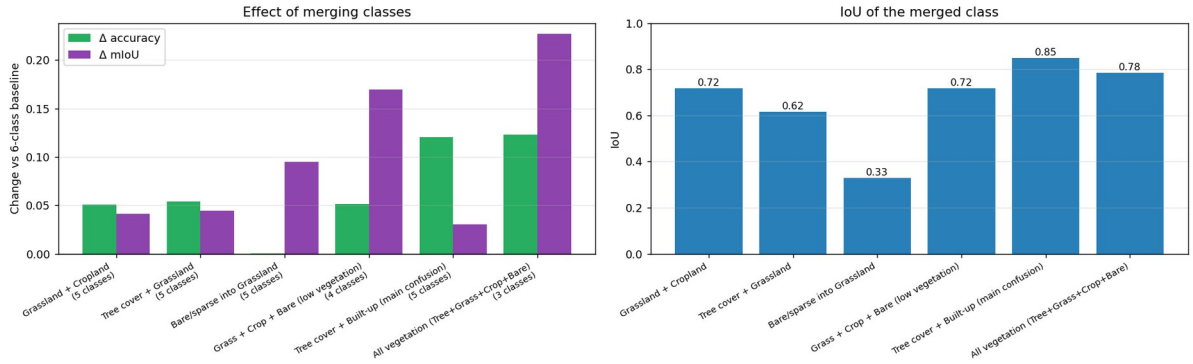


Figure 13. Class merging: change in accuracy and mIoU (left) and IoU of the merged class (right) (notebook 04).

9. Discussion and limitations

9.1 What the model has learned

The results fit together well. The network separates classes that scatter the radar signal in different ways: mirror-like water, double-bounce buildings, volume-scattering forest and fields that change through the season. It struggles where these mechanisms overlap. Its use of the VH/VV relationship for cropland and of spatial context for all classes matches what radar physics would predict. The confusion between trees and buildings has two causes. In suburbs and the city centre, trees and buildings share the same 10 m pixels and the building echo wins. In addition, dense forest on slopes that face the satellite can return a strong signal. Grassland sits between trees and fields and has no clear signature of its own at C-band, which explains its low IoU.

9.2 Limitations

- **Static reference labels.** WorldCover 2021 is used as the label for every month of 2025–26. A disagreement can come from a model error, a real change since 2021 or an error in WorldCover, and none of the scores is an accuracy against field data.
- **Seasons versus change.** The model sees one month at a time, so seasonal changes in the radar signal show up as class changes. The 20 km analysis shows swings of several percentage points that are not physically plausible. Changes of a few percentage points in stable areas should be treated as noise.

- **One region.** All data come from around Stuttgart and from descending passes. The prototype accepts any coordinate, but how well it works in other landscapes, other climates or on ascending passes has not been tested.
- **Rare classes.** Bare/sparse vegetation is never predicted, and its IoU of zero lowers the mIoU by about 0.10.
- **Model selection.** The validation set used to pick the checkpoint has a very different class mix from the test set, so the validation scores overstate performance.
- **Map colours.** In the map colour scheme, grassland and cropland look very similar and are hard to tell apart, especially for readers with colour-vision deficiencies. The charts in this report therefore name the classes directly.
- **Gaps in the documentation.** The training curves, the original data-preparation code, the class statistics of the training set and the orbit metadata are not in the repository (Appendix B).

10. Conclusion and future scope

A small U-Net trained only on Sentinel-1 VV and VH backscatter reaches 0.719 pixel accuracy and 0.473 mIoU against WorldCover on held-out locations around Stuttgart. It recognises water, cropland and built-up land reliably, but has clear weaknesses for grassland and cannot detect the rare bare/sparse class. The ablation study shows that the model uses both polarisations, depends on their relationship to find cropland, and relies mainly on spatial context. The confusion between tree cover and built-up is the largest single source of error. The trained model runs in a working prototype that produces twelve monthly maps and change statistics in percentage points for any location, with checks for orbit consistency and valid inputs.

10.1 Future work

5. **Retrain with four classes** (tree cover, low vegetation, built-up, water). The merging analysis suggests this would reach an mIoU of about 0.64 with classes that are still meaningful.
6. **Use several months as input**, for example three consecutive months, so the model can tell seasonal changes in the radar signal apart from real changes in land cover.
7. **Add more varied regions**, including both orbit directions, to test and improve how well the model transfers.
8. **Add data augmentation and regularisation** (flips, rotations, speckle noise) and a learning-rate schedule, since the best epoch came early.
9. **Use better reference data**, such as more recent or independently checked labels, for example Dynamic World [14] or local land registry data, to separate model errors from label errors.
10. **Evaluate the prototype again** with overlapping tiles at more sites, and show the uncertainty of each pixel.

References

[1] R. Torres et al., "GMES Sentinel-1 mission," Remote Sensing of Environment, vol. 120, pp. 9–24, 2012.

[2] D. Zanaga et al., "ESA WorldCover 10 m 2021 v200," Zenodo, 2022, doi: 10.5281/zenodo.7254221.

[3] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. MICCAI, LNCS 9351, pp. 234–241, 2015.

[4] N. Gorelick et al., "Google Earth Engine: Planetary-scale geospatial analysis for everyone," Remote Sensing of Environment, vol. 202, pp. 18–27, 2017.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. ICLR, 2019.

[6] J. Long, E. Shelhamer and T. Darrell, "Fully convolutional networks for semantic segmentation," in Proc. IEEE CVPR, pp. 3431–3440, 2015.

[7] F. T. Ulaby and D. G. Long, Microwave Radar and Radiometric Remote Sensing. Ann Arbor: University of Michigan Press, 2014.

[8] A. Veloso et al., "Understanding the temporal behavior of crops using Sentinel-1 and Sentinel-2-like data for agricultural applications," Remote Sensing of Environment, vol. 199, pp. 415–426, 2017.

[9] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[10] P. Olofsson et al., "Good practices for estimating area and assessing accuracy of land change," Remote Sensing of Environment, vol. 148, pp. 42–57, 2014.

[11] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019.

[12] J.-S. Lee, "Digital image enhancement and noise filtering by use of local statistics," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 2, no. 2, pp. 165–168, 1980.

[13] Google Earth Engine Data Catalog, "Sentinel-1 SAR GRD: C-band Synthetic Aperture Radar Ground Range Detected, log scaling" (COPERNICUS/S1_GRD). https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S1_GRD

[14] C. F. Brown et al., "Dynamic World, near real-time global 10 m land use land cover mapping," Scientific Data, vol. 9, art. 251, 2022.

Appendix A. Reproducibility

The code, trained weights, selected results and notebooks are at https://github.com/Mehatab1/DeepLearning. The README lists every command; these are the main steps:

```
git clone https://github.com/Mehatab1/DeepLearning.git sentinel1-lulc && cd
sentinel1-lulc
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
earthengine authenticate     # or: python -c "import ee;
ee.Authenticate(auth_mode='notebook')"
export EE_PROJECT=<your Google Cloud project>
pytest -q                    # 16 offline tests
python run_lulc.py --lat 48.7758 --lon 9.1829 --start 2025-10 --area-km 2
streamlit run app.py
python scripts/check_accuracy.py --project $EE_PROJECT
```

Table 11. Where the results in this report come from.

Result	Source in the repository
Training configuration, validation metrics and confusion matrix	notebooks/archive/ 09_train_unet_v5_expanded_area.ipynb
Test-set metrics (per class, per month, confusion matrix)	results/accuracy/test_set/
Prototype run, south-west of Stuttgart	results/accuracy/48.6900_9.0600_2025-10_5km/
Prototype run, Frankfurt	results/accuracy/frankfurt_2025-10_to_2026-09/
20 km Stuttgart analysis	notebooks/archive/10_v5_lulc_prototype.ipynb, results/figures/
Feature ablation, class removal, class merging	notebooks/04_ablation_study.ipynb, results/ablation/
Trained model and normalisation statistics	weights/best_unet_v5.pt

Appendix B. Open items and data availability

The items below were not available when this report was written. They are listed here instead of being filled in with guesses.

Table 12. Open items.

Item	Status / action
Dataset download link	Not published yet. Build the archive with python scripts/package_dataset.py and upload it, e.g. to Zenodo.
Training curves	Only on the lab server. Re-run notebook 02 and add them as Figure 6a.
Notebook outputs (01–03)	These notebooks are stored without outputs; notebook 04 has its outputs.
Original data-preparation code	Not in the repository; notebook 01 rebuilds the procedure.
Training-set class distribution, orbit numbers, scene counts	Not recorded.
Coordinates of validation and test locations	Not documented.

Item	Status / action
Stuttgart-centre prototype runs (Table 7)	Values taken from console output; the result files were overwritten.
Frankfurt evaluation script	Not in the repository.
Screen recording of a full run	Still to be recorded (plan in docs/RECORDING.md).